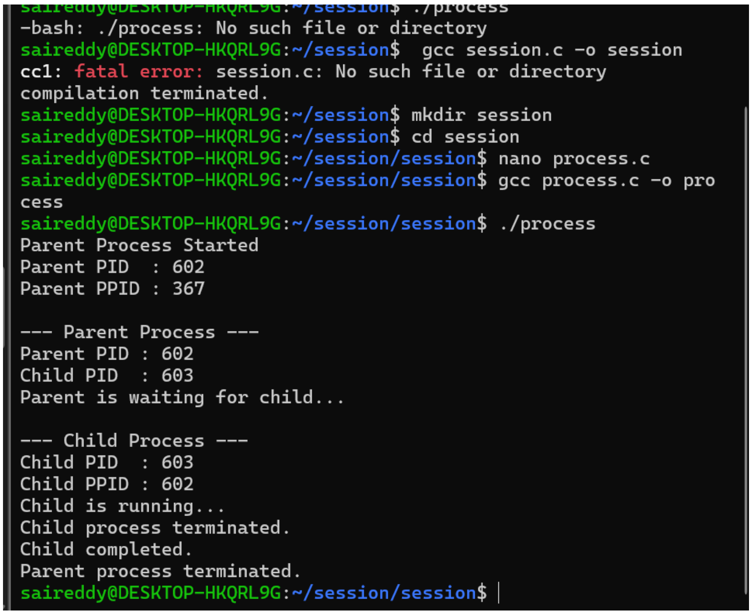
Create the C program using fork()




```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>


int main() {

   pid_t pid;


   printf("Parent Process Started\n");

   printf("Parent PID  : %d\n", getpid());

   printf("Parent PPID : %d\n", getppid());


   pid = fork();


   if (pid < 0) {

      perror("fork failed");

      return 1;

   }
```

```c
    if (pid == 0) {
        // Child process
        printf("\n--- Child Process ---\n");
        printf("Child PID  : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is running...\n");
        sleep(10);

        printf("Child process terminated.\n");
        exit(0);
    }
    else {
        // Parent process
        printf("\n--- Parent Process ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        printf("Parent is waiting for child...\n");
        wait(NULL);

        printf("Child completed.\n");
        printf("Parent process terminated.\n");
    }

    return 0;
}
```

```
saireddy@DESKTOP-HKQRL9G:~/session/session$ gcc fork.c -o fork
cc1: fatal error: fork.c: No such file or directory
compilation terminated.
saireddy@DESKTOP-HKQRL9G:~/session/session$ ./fork
-bash: ./fork: No such file or directory
saireddy@DESKTOP-HKQRL9G:~/session/session$ nano fork.c
saireddy@DESKTOP-HKQRL9G:~/session/session$ gcc fork.c -o fork
saireddy@DESKTOP-HKQRL9G:~/session/session$ ./fork
Parent Process
Parent PID : 640
Child PID  : 641
Parent is waiting for child...
Child Process
Child PID  : 641
Child PPID : 640
Child is running...
Child process terminated.
Child completed.
Parent process terminated.
saireddy@DESKTOP-HKQRL9G:~/session/session$
```

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>


int main()

{

   pid_t pid = fork();


   if (pid < 0)

   {

      printf("Fork failed\n");

   }

   else if (pid == 0)

   {

      printf("Child Process\n");

      printf("Child PID  : %d\n", getpid());

      printf("Child PPID : %d\n", getppid());


      printf("Child is running...\n");

      sleep(5);
```

```
        printf("Child process terminated.\n");
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        printf("Parent is waiting for child...\n");
        wait(NULL);

        printf("Child completed.\n");
        printf("Parent process terminated.\n");
    }

    return 0;
}
```